Approach D — Target-Conditional Boltzmann Diagnosis and Relative-Motion Rearrangement

Methodology and GitHub Migration Specification | 2D-first research build

Research implementation specification

28 October 2026 | Proposed v1.0

STATUS AND AUTHORITY. This is a **proposed Approach D methodology and migration contract**, not a description of completed experimental evidence. Its baseline is the current `Narendhiranv04/latent-causal-tamp` repository (`p17_d.py`, `d_blockers.py`, `p14_sweep_index.py`, `p15_lookup.py`, `p16_motion.py`, `p17_domain.py`, and `p17_search.py`) and P17 reports. It intentionally modifies the existing D implementation. It supplements, rather than silently rewriting, the uploaded A/B/C methodology v1.7 and implementation guide v1.9. **Do not claim the proposed pipeline has been tested until its independent experiments are run.**

1. Research decision: one learned task, one geometric task

Fixed scientific question. Can a scene-conditioned Boltzmann blocker diagnosis improve the **efficiency and quality of selecting relocations that restore feasibility of the *original* target action**, compared with a strong geometry-only rearrangement search using the same motion library and budget?

The system receives a scene S_0, a fixed-base articulated robot configuration q_0, one **original target** T, and a specified target destination g_T. Its output is a short, *ordered* sequence of feasible relocations $r = [(o_1, d_1), \dots, (o_m, d_m)]$ and supporting verification evidence, such that the original target action becomes executable after the relocations.

Architecture contract:

1. **Learned Approach D:** Given *only* (S, q, T, g_T), predict (a) probability the original target action is feasible and (b) which movable objects participate in making the original target task infeasible. After a proposed relocation, query the **same model for the same original target** T in an updated hypothetical scene. Never rebind obstacle A/B/C as a substitute target in the production D policy.
2. **Non-learned relative-motion retrieval:** Generate candidate destination cells; retrieve locally analogous stored **relative object relocations** using 3×3 and 4×4 patches, not absolute source/destination keys; adapt candidate motion patterns to the new robot-relative workspace position; check the actual trajectory and swept occupancy in the current scene.
3. **Non-learned rearrangement search:** Select obstacle/destination/order using declared deterministic heuristics and bounded search. The Boltzmann graph **does not** predict relocation order, destination, grasp or trajectory. A geometric oracle establishes motion admissibility; the D score is a heuristic, never a proof.
4. **Independent final confirmation:** A high D feasibility prediction is not a verified solution; a task verifier must validate the original target after the proposed relocation prefix. Planner exhaustion, missing witnesses and incomplete evidence remain `UNKNOWN`, never proven infeasible.

Main simplifying assumption: in the primary 2D suite, obstacles contributing to the original target blockage generally have at least one available, directly reachable relocation cell. Nested physical dependencies (B must be moved solely to make moving A possible), recursive blocker-of-blocker diagnosis and repeated relocation of the same object are **out of the primary scientific scope**. They can be recorded as stress tests, not silently counted as failures of the causal-diagnosis hypothesis. This is a declared *dataset scope*, not a universal manipulation assumption.

1.1 Explicitly retained and removed

Retain	Change / remove from production D
Frozen DINOv2-small pooled appearance, geometry, target/robot context, two output heads.	Do not query D with an obstacle as temporary target (`Guide.order` recursion).

Retain	Change / remove from production D
Feasibility BCE + blocker negative log-likelihood; **no third `L_updated`**. Intervention-grounded blocker labels and complete/unknown status tracking. Historical paired D as an ablation; pairless Dp17 as primary learned model. Multiple stored motion modes, arm-continuity checks, online-planner fallback. Non-learned geometry-only and no-recursion controls.	Add valid post-relocation target-only scene examples as **more samples under the same losses**, where necessary. Do not require enumerating minimal-set unions **at inference** or moving every object in the predicted blocker union. Do not interpret quadratic edges as directional physical dependencies or change the architecture simply to model rare B-blocks-A cases. Replace absolute `(src,dst)` motion retrieval with **relative displacement + localized occupancy matching**. Explicitly compare nearest-to-robot ordering, motion-cost ordering and D-guided ordering under equal budgets.

2. Formal definition and causal scope

Let the state be $S = (\{p_i, h_i\}, \mathcal{O}, \mathcal{F}, q)$ with object poses/geometries, occupancy, fixed structures and full robot joint configuration. Let $Y_T(S, q, g_T) \in \{0, 1\}$ denote feasibility of the *specified original target action* under a standardized pickup, transport, placement and retreat contract. For ground truth, use `VERIFIED_FEASIBLE`, `CERTIFIED_INFEASIBLE`, or an explicitly typed `UNKNOWN`. **A failed bounded planner call does not turn into 0.**

A diagnostic intervention $I = \text{do}(p_A \leftarrow d_A)$ is an admissible scene edit: A is placed at a candidate position, other objects and the task are held fixed, and the original target action is evaluated. This edit tests how relocating A affects Y_T; the edit **does not by itself prove that the robot can execute the relocation**. Distinguish training's fixed-q pose edits from runtime's reachable post-motion q' states.

For a small pool $\mathcal{M}$ of eligible movable objects, we can diagnose minimal sufficient clearing sets in a *declared finite intervention domain* $\mathcal{J}$. For example:

- $Y_T(S_0) = 0, Y_T(S_0^A) = 1$: relocating A to some admissible tested cell is sufficient.
- $Y_T(S_0^A) = 0, Y_T(S_0^B) = 0, Y_T(S_0^{A,B}) = 1$: A and B together are sufficient in that intervention setting.
- If either singleton A **or** singleton B clears T, minimal sufficient sets are $\{A\}$ and $\{B\}$; **both need not be moved at inference**.

The training blocker mask b^* is the **union-participation indicator** across known inclusion-minimal sufficient clearing sets (not the list of universally necessary objects). Its semantics are *intervention-domain specific*. Save `minimal_sets` and `necessary_intersection` separately for audits, but **do not make runtime inference enumerate the entire intervention lattice**. For incomplete, undecided, non-monotone or no-sufficient-set records, mask the blocker loss; keep feasible originals as supervised feasibility examples. Do not manufacture an all-zero blocker label from missing evidence.

Scientific limitation: an inferred high blocker probability is statistical evidence learned from counterfactual labels. It is **not** certification that an object is universally necessary, nor a physical explanation of which link collides with which obstacle. The actual effect of a proposed edit is independently checked.

3. Conditional Boltzmann model and exact losses

3.1 Input, features and outputs

Reuse `d_blockers.ModelD` and the existing scene encoder (`abc_models.Root`): object geometry and position, frozen mask-pooled DINOv2-small features, target token, global pooled objects, robot configuration/kinematics and target-goal context. **Forbidden leakage:** teacher trajectory, target swept-volume collision labels, privileged intervention outcome, future scene, root/family ID or ground-truth blocker set at inference.

For the original target T only, let the shared learned network f_θ output:

- **Feasibility logit** $u_\theta(S, T, g_T)$, giving $p_T = \sigma(u_\theta)$.
- **Blocker unary logits** $a_{i,\theta}(S, T, g_T)$ for each eligible movable object i.
- Optionally pairwise scores $w_{ij,\theta}$ **only for a historical/ablation comparator**; production Dp17 is `pair=False`.

The original paired conditional blocker energy is

$$E_\theta(b \mid S,T) = -\sum_{i\in\mathcal{M}} a_i b_i - \sum_{i<j} w_{ij} b_i b_j, \qquad b_i \in \{0,1\}.$$

Its conditional Boltzmann probability is

$$P_\theta(b \mid Y_T = 0, S, T) = \frac{\exp[-E_\theta(b \mid S,T)]}{Z_\theta(S,T)}, \quad Z_\theta = \sum_{b'\in\{0,1\}^{|\mathcal{M}|}} \exp[-E_\theta(b')].$$

In **production pairless D** ($w_{ij} = 0$), this factorizes:

$$P_\theta(b \mid Y_T = 0, S, T) = \prod_i \sigma(a_i)^{b_i} \left(1 - \sigma(a_i)\right)^{1-b_i}.$$

This model is a **conditional energy model**; with no pair factors, its blocker outputs are effectively independent Bernoulli probabilities conditioned on the shared scene features. Do **not** claim the deployed model has learned physical pairwise relations. Keep the paired variant for the controlled paired-vs-pairless ablation only.

Because the blocker probabilities are conditional on infeasibility, define the *unconditional prioritization score*

$$\gamma_i = (1 - p_T)\, P_\theta(b_i = 1 \mid Y_T = 0, S, T).$$

Use γ_i only to prioritize candidate interventions, not to certify impossibility or prune all lower-scored candidates.

3.2 Training labels

For each **original-target** training query: store `scene_root`, `split`, `scene_hash`, `(T,g_T,q)`, `status_original`, eligible intervention pool, eligibility constraints, tested scene edits with destination assignments and statuses, minimal sufficient sets (when decidable), union mask b^* (when defined), necessary intersection (audit only), and provenance. For multiple destinations, a subset is sufficient if at least one admissible assignment makes T verified feasible; it is insufficient only if every assignment is certified negative; otherwise `UNKNOWN`.

Split by scene root and geometry fingerprint **before generating intervention derivatives**. A post-edit scene from a test root must never appear in train or calibration. Track root and label coverage rather than only the number of edited queries. Include partial-clearance scenes where T remains infeasible after relocating one of two necessary blockers.

3.3 Two losses only

Original-task feasibility BCE:

$$L_{\text{feas}}(S) = -y_T \log p_T - (1 - y_T) \log(1 - p_T).$$

When a **proof-grounded, fully scoped** blocker mask exists for an infeasible original:

$$L_{\text{blocker}}(S) = -\log P_\theta(b^* \mid Y_T = 0, S, T).$$

Final training objective:

$$\boxed{L_D = \mathbb{E}_S \left[L_{\text{feas}}(S) + \lambda_B \mathbf{1}_{\{y_T=0,\ b^* \text{ valid}\}} L_{\text{blocker}}(S) \right].}$$

Preserve $\lambda_B = 2$ as the P17 starting configuration and run controlled sensitivity if needed. The model's parameters (the feature encoder's trainable projections and unary/feasibility heads) are updated by AdamW on this objective. The frozen DINOv2 feature extractor is not finetuned by default. Retain validation-based early stopping, three seeds, input normalization and existing checkpoint provenance. **No loss for candidate destination, motion cost, relocation order, updated-scene category or artificial edge labels.**

Updated hypothetical scenes need no new loss. They are simply more (S, T, g_T, y_T, b^*) samples using the same equations. When post-relocation q' changes, collect some train-only target queries using achieved robot configurations to guard against distribution shift; use feasibility BCE even if a complete blocker mask cannot be obtained. Record the sample's intervention protocol and do not leak test-derived intermediate states.

4. Relative swept-motion retrieval: the required migration

4.1 Existing behavior and why it must change

The current `p14_sweep_index.py` stores `delta_rc`, 3×3 and 4×4 source/destination descriptors and whole-scene occupancy, **but indexes candidates by exact `(src,dst)`**. This excludes a useful motion pattern at cells `(10,18)` from a query at `(34,42)` despite identical relative displacement $(\Delta r, \Delta c) = (1, 0)$. The existing local similarity windows are **fixed patches at endpoints**, not a literal CNN convolution over every whole scene. Indexed, precomputed patches are sufficient and cheaper.

Required new retrieval key: normalized object geometry class / footprint and relative displacement $(\Delta r, \Delta c)$, **not absolute cells**. Also store robot-relative start and goal position to help rank/stratify the reachable subset of matches (a fixed-base arm is not translation invariant).

4.2 Stored entry and query descriptor

For each **provenance-valid real motion witness** τ_k store:

$$K_k = (\Delta r, \Delta c, \text{object footprint}), \quad F_k = (W_s^3, W_s^4, W_d^3, W_d^4, G, \rho_s, \rho_d),$$

where W are categorical occupancy patches **centered/anchored consistently at the motion's source or destination**, G optionally captures weak global occupancy context, and ρ_s, ρ_d summarize distances/angles **relative to the fixed robot base**. Include recorded joint path, gripper/contact/attachment phases, terminal q, derived swept cells, path cost, support scene/robot IDs and the exact verification provenance. Store one canonical descriptor per indexed witness; no runtime rescan of all videos or images.

For a query $A : p \to d$, compute its K_q, F_q from the **current** scene. Retrieve witnesses under a compatible relative displacement and geometry class. Rank by, for example,

$$R(q,k) = \alpha_3 \,\mathrm{sim}(W_q^3, W_k^3) + \alpha_4 \,\mathrm{sim}(W_q^4, W_k^4) + \alpha_\rho \,\mathrm{sim}(\rho_q, \rho_k) + \alpha_G \,\mathrm{sim}(G_q, G_k).$$

This is a **deterministic retrieval score, not a learned D energy**. Initial coefficients should reproduce the previous local/global ranking as closely as possible, with the new robot-relative term isolated in an ablation. Avoid claiming that a fixed heuristic is optimal. Test patch boundary padding, coordinate frames, translation consistency and invariance to object IDs.

4.3 Retrieve motion modes, not thousands of near duplicates

The library may contain many witnesses with the same relative displacement. Group candidate witnesses into modes by swept-shape similarity **and compatible terminal robot configuration**, using deterministic canonical sorting and per-source provenance. For performance, precompute mode descriptors and a compact shortlist. Rank **modes first**, then a small number of witnesses within each mode. Example initial configurable budget (engineering default to validate, **not a proven optimal setting**): 3 distinct modes × at most 2 witnesses/mode, then optionally 1 bounded online planning attempt; use a separate outer query-level limit so source/destination trials cannot grow unchecked.

Essential policy: stop checking witnesses for a destination after finding a valid motion if the objective is first admissible relocation. If comparing costs, inspect a declared additional budget. Do **not** mark a destination physically impossible because a witness ranking threshold or bounded shortlist found nothing. Report `UNKNOWN_NO_WITNESS`, `UNKNOWN_ADAPT`, `UNKNOWN_BUDGET`, `VERIFIED` and truly sound, scoped `CERTIFIED_INFEASIBLE` separately.

4.4 Fixed-base robot constraint: relative *object* motion is not translated joint motion

A fixed-base 3R arm does not have translation symmetry. **Do not simply shift the stored joint trajectory or its swept cells and call it valid.** The same relative displacement at a different absolute position may require different joint angles, grasp conditions or even be unreachable.

A permissible concrete adaptation pipeline is:

1. Extract from the stored witness a task-space end-effector path $x_k(t) = \mathrm{FK}(q_k(t))$, with grasp/attachment phase boundaries.
2. Express its waypoint geometry relative to the *stored* object source, then align this local movement pattern with the query's *new* object source and intended destination. Keep orientation/grasp conventions consistent; translations/rotations require explicit supported symmetry.

3. Solve **continuity-preserving IK** at the new workspace coordinates (or warm-start the existing planner from the retrieved path's *shape*). Joint limits, grasp/transport feasibility and current-start-to-pregrasp transit remain mandatory.
4. Reconstruct the robot's **actual** swept volume from the adapted joint trajectory, not from a blindly translated stored swept mask. Check all link, gripper, carried-object, approach, carry, placement and retreat collisions against the current scene.
5. If adaptation fails, try a distinct mode/witness or the existing bounded online skill planner. If no candidate certifies, the result is `UNKNOWN` unless an independent sound impossibility certificate exists.

This adaptation operation **is a new engineering component**. It does not currently exist as a generally valid translation of the stored trajectories. Preserve the original exact-cell lookup as a comparison and fallback during migration.

5. Runtime D policy: target-only iterative counterfactual diagnosis

5.1 Candidate selection and ordering

At every hypothetical search state $X_k = (S_k, q_k, r_{1:k})$, call **D only with the original target** (T, g_T). Use the predicted probabilities $p_T(X_k)$ and $\gamma_i(X_k)$ to rank eligible suspected blockers.

The simplest policy aligned to the intended dataset is **likely blockers first; nearest to the *current robot end effector* first among them**. Use estimated retrieved travel as the next tie-breaker. Keep a geometry-only comparison that uses exactly the same ordering without γ_i. Also evaluate direct minimum estimated motion-cost order. Robot-base distance, target distance and end-effector distance are *different* heuristics; record which one is used. **The Boltzmann machine does not learn order.**

Never permanently discard a lower-probability blocker solely because D gave it a low score: use top-K soft prioritization followed by a fixed widening rule when no solution is found. This prevents false-negative diagnosis from making the method silently incomplete.

5.2 Hypothetical relocation loop

For selected object A:

1. Generate candidate free destination cells: annulus-safe, unoccupied, not the original target/goal, with simple geometric separation. Use proximity/cost to order slots; retain multiple destinations, including any newly freed cells if permitted.
2. Query the new **relative-motion** index for (A, p_A, d_A); retrieve a bounded, diverse motion shortlist; adapt and independently verify the chosen relocation in S_k from q_k. If a particular path is blocked, **try alternatives; do not immediately delete the entire destination**.
3. Construct the hypothetical next scene $S_{k+1} = \text{Apply}(S_k, A \mapsto d_A)$ and set q_{k+1} to the **verified trajectory's terminal robot configuration**. This state is counterfactual at planning time, not an assumption of real-world execution.
4. Query **the same D network for T again**, not for A. Obtain $p_T(X_{k+1})$ and updated blocker marginals. If the candidate looks promising, independently verify the target task. A verified target task yields a candidate complete corrective plan.
5. If T remains blocked, keep promising intermediate states and consider another object or destination. A may no longer appear as a blocker while B remains; **do not require p_T to rise after every individual move**. Jointly necessary A+B situations can require two moves before T becomes feasible.
6. Limit recursion by a finite number of *relocations*, state/trajectory queries and distinct hypothetical states. Use a small beam or bounded best-first strategy to retain alternatives; do not greedily accept every predicted progress step without a rollback path. In the primary dataset, prefer one move per object unless a separately labeled stress test requires repeats.

Crucial runtime distinction: the **blocker list need not become empty** as an extra success test if the target itself is verified feasible; the predictor can have false positives. Conversely, simply removing A from a predicted blocker set **is not** evidence that the original target has become feasible. D's new state estimate guides planning; the independently checked target outcome decides final success.

5.3 Search objective, stopping and statuses

Use lexicographic optimization over the explored, physically verified repair domain:

$\min_r \left(N_{\text{relocations}}(r),\ J_{\text{joint travel}}(r)\right)$ subject to verified relocation prefixes and verified original target action.

Nearest-robot-first, blocker-score-first, similarity and cheapest-motion-first are *expansion heuristics*, not the definition of the optimum. Distinguish **first verified within budget** from **proven minimum within a completely adjudicated finite graph**. A timeout, insufficient lookup coverage or failed online planner remains `NO_PLAN_FOUND/UNKNOWN`, not a certificate that no repair exists.

Minimal policy pseudocode (proposed; not current behavior):

```
search(state0, original_target=T, goal=gT):
    frontier = [(state0, q0, empty_repair)]
    while frontier and budget_remaining:
        state, q, repair = pop_best(frontier)
        D = predict_D(state, q, T, gT)          # original target ONLY
        if verify_original_target(state, q, T, gT) == VERIFIED:
            return VERIFIED_PLAN(repair)
        movers = eligible_objects_ranked_by_D_then_robot_distance(D, state, q)
        for A in movers:
            for cell in candidate_free_cells(A, state):
                motion = relative_lookup_adapt_verify(A, cell, state, q)
                if motion.status != VERIFIED:
                    record_status(motion.status); continue
                state2, q2 = apply_hypothetically(state, motion)
                add_state_with_cycle_and_budget_controls(frontier, state2, q2,
                                                         repair + [(A, cell, motion)])
    return NO_PLAN_FOUND_WITH_EXPLICIT_UNKNOWN_FLAGS
```

Calling D at a node **before** an optional verification of T is the required learned re-query; do not short-circuit into declaring success from prediction alone. A solver may cache target-model outputs by $(\text{scene fingerprint}, q, T, g_T)$ and must count the virtual work of a cache hit fairly across arms.

6. Proposed minimal configuration and code migration

Do not rebuild the repository. Work incrementally atop existing P14–P17 code; retain historical tests and six replay fixtures. Proposed minimal modifications:

Existing file	Intended narrow change
`p14_sweep_index.py`	Add relative-displacement indexing and aligned patch descriptors **without deleting** absolute-index history. Add provenance-preserving mode metadata.
`p15_lookup.py`	Relative top-mode retrieval and benchmark; preserve leakage exclusions and compare against the legacy exact-cell lookup.
`p16_motion.py`	Add a switchable relative witness adapter with certified new-scene trajectory validation; preserve legacy exact-cell and online fallback; no claim that translated joint paths are valid.
`p17_domain.py`	Candidate destination generation; consistent geometric rules and deterministic nearer-to-robot / travel-cost rankers.
`p17_d.py`	Add `Dp17-targetonly` guide: prohibit temporary-target D calls, re-evaluate **T** on new hypothetical scene states, use blocker scores as soft guidance. Keep `Dp17` and `Dp17-norec` as frozen historical arms.
`p17_search.py`	Reuse finite graph, budget and replay semantics; minimally expose the needed node-level guidance/verification hooks.
`d_blockers.py`	**Do not change** primary mathematical model or two-loss objective without a failing correctness test.

Add **at most one** focused module, e.g. `relative_motion.py`, for end-effector waypoint alignment + IK adaptation if it cannot remain comprehensible inside the existing motion code. Prefer short, orthogonal functions; no duplicated planner, robot model or second data format. New config keys must be in existing TOML/JSON infrastructure and logged in the experiment manifest. Avoid broad directory reshuffles.

Illustrative initial knobs, to preregister then validate:

```toml
[approach_d]
policy = "target_only"
requery_original_target = true
obstacle_as_temporary_target = false
pair_factors = false
soft_blocker_top_k = 3
ordering = "blocker_then_nearest_ee"
max_relocations = 3
beam_width = 8

[relative_lookup]
key = "delta_rc_and_footprint"
patches = [3, 4]
mode_limit = 3
witnesses_per_mode = 2
online_fallback = true
allow_absolute_fallback = true
status_on_no_witness = "UNKNOWN"
```

These are **starting configurations for experiments**, not results or final hyperparameters. Respect existing 2D safety checks and keep physical budget parity across all comparison arms. If key names conflict with existing config, introduce mapped equivalents rather than parallel configuration authorities.

7. Stepwise implementation gates (2D first)

G0 — Repository audit and frozen baseline. Inspect the current branch/commit, status and project docs. Trace all D model calls and all absolute lookup keys. Produce an explicit current-vs-proposed call graph and a file/function patch list. Run all existing tests and six historical replays. **Gate:** baseline reproducible; no code changes before audit report.

G1 — Relative-only retrieval (no adaptation yet). Build $(\Delta r, \Delta c, \text{footprint})$ index, patch descriptors, deterministic mode grouping and leakage-safe provenance. Demonstrate two translated local scenes map to the same relative key and can retrieve each other's witnesses. Keep exact-cell control. **Gate:** indexed retrieval correctness and held-out neighbor transfer metrics, including counts and efficiency.

G2 — Motion adaptation and geometric validation. Implement end-effector path alignment/IK continuity, attachment-phase preservation, actual new-location swept volume reconstruction, final end-pose and transit validation. Retain alternate modes and planner fallback. **Gate:** (a) successful cross-location replay of real relocation, (b) collision rejected for link/carry/gripper mid-path, (c) unreachable translated action not falsely certified, (d) `UNKNOWN` preserved.

G3 — Target-only D inference loop. Add a new arm rather than rewriting historical `Dp17`: D calls exclusively reference T; validate hypothetical (S', q') input rebuilding; re-query T after each proposed relocation; use blocker priors and deterministic nearest-end-effector ordering; compare alternative placement cells. **Gate:** instrumented unit test proves zero calls with `target != T`, and one or more re-queries with changed scene/q and same T.

G4 — Training-data coverage audit; retrain only if needed. Quantify original and reachable post-relocation target-only scene distribution; verify paired labels and root isolation. If intermediate states are underrepresented, add appropriately labelled training scenes using the **existing** $L_{\text{feas}} + \lambda_B L_{\text{blocker}}$; no new loss. Do not include test roots. **Gate:** held-out original-target and post-edit target feasibility/calibration both reported.

G5 — Paired scientific decision. Freeze the scene roster, candidate placements, motion limits and verification backend. Compare the new D arm against strong geometry-only baselines, nearest-end-effector ordering, estimated-motion-cost ordering, existing `Dp17-norec`, and relative-vs-absolute lookup on the same scenes. Three seeds for learned arms; count actual and virtual D calls, retrieved modes, physical checks and CPU. **Gate:** report effect sizes, discordant paired roots and uncertainty; a negative result is acceptable.

G6 — Only after stable 2D evidence, transfer to simple 3D. Preserve conceptual contracts (original target only; relative/local retrieval; kinematic adapter; full collision verification). Rebuild descriptors around 3D translational/rotational transformations with appropriate task-frame invariances, not naive 2D grid shifting. Do not claim

3D generalization from 2D-only tests.

8. Required tests and validation fixtures

Lookup / geometry: identical Δ but different absolute cells retrieve same relative family; distinct Δ not mixed; 3×3/4×4 source/destination windows aligned; border padding consistent; objects with wrong footprint rejected; relative descriptors insensitive to object ID; source/goal local patches tested separately; same delta but different robot-base radius does not bypass IK; new-location swept volume recomputed; different mode can succeed when top mode fails; no lookup hit returns `UNKNOWN` rather than infeasible; no train/eval witness leakage by root **or geometry fingerprint**.

Model: pairless partition and marginal correctness; feasibility BCE + masked blocker NLL only; no blocker term on unknown/no-clearing; no invented all-zero masks; alternative singleton minimal sets yield union but runtime permits one-move success; two jointly necessary blockers require candidate prefix depth ≥2; original target fixed through all re-queries; non-original-target D calls **zero**; q after motion carried into next target query; predicted blocker disappearance alone never declares verified success.

Search: distinct destination choices tested; a single blocked witness does not eliminate a destination; one verified path can stop further witness checks in first-admissible mode; beam can backtrack when one destination looks promising but later fails; no false negative from hard blocker-threshold gating; cycle/duplicate-state suppression; partial failure status logged; min-moves/travel proof only under complete finite adjudication; stable independent physical replay. Retain all baseline tests and archived-run replay fixtures.

Visualization per phase: full 8×8 scene and articulated robot, patch locations (3×3, 4×4), relative displacement vector, top retrieved motion modes, original vs adapted swept volumes, first actual collision, candidate destination overlay, D original-target blocker probabilities before/after edits, beam/search tree, relocation order, actual verified robot motion GIF. Clearly label **predicted**, **hypothetical**, **verified**, **UNKNOWN**, and **not yet tested** states. Never draw an imagined executable motion as evidence.

9. Ablations, metrics, and honest claims

Comparison	Hypothesis isolated
New D target-only vs geometry-only, **same relative backend**	Does learned blocker diagnosis improve repair selection?
D target-only vs existing Dp17-norec, **same backend**	Does updated-state use/policy differ from historical state re-query and ordering?
Relative vs exact-cell lookup, **same search & budget**	Does local translational retrieval improve cross-location coverage?
3×3 / 4×4 / both / global-context term	What local context is necessary for witness ranking?
Diverse mode shortlist vs all/top-1 witness checks	Retrieval efficiency vs valid-motion coverage.
Nearest-end-effector vs estimated travel vs blocker-score-first	Is geometric or learned action ordering responsible for gains?
D queried after each hypothetical edit vs original-only one-shot	Does updated target reassessment add value?
Original-only training vs original + intermediate **same-loss** examples	Is updated-state data coverage limiting diagnosis accuracy?
Pairless vs paired D retrained on matched data	Are learned quadratic blocker interactions useful?

Report **separately**: original-target AUROC/Brier/feasible and infeasible recall; blocker precision/recall/F1/exact mask on defined proof-grounded labels; held-out intermediate-state calibration and change prediction; retrieval recall@K, witness-to-new-location adaptation rate, modes and exact checker calls; verified corrected-task success, corrections per trial, joint travel, target checks, CPU and total node count; fractions of `UNKNOWN` and soundly certified outcomes; ablations with paired root-level confidence intervals and three learned seeds. Keep strict success as an **independently verified** full relocation+target plan, not a neural probability threshold.

Historical context, not evidence for the new design: P17 prospective Dp17 solved **70/90**, Dp17-norec **71/90**, geometry-norec **71/90**. Original-target Dp17 had AUROC **0.952** and blocker F1 **0.893** on the 31-mask parking evaluation. The current D search's success is largely attributable to the shared physical search, not a proven learned ordering advantage. This proposed relative adaptation and target-only decision rule have **not** yet been run; do not apply the historical scores to them.

Predeclared acceptance: scientific support requires matched-budget, prospective root-level improvement with uncertainty reported. Strong offline blocker accuracy alone is insufficient. If new D remains equivalent or worse than geometry-only, report the result and simplify rather than adding more model heads/losses or selectively changing the benchmark.

10. Deliverables and non-negotiable engineering standards

Expected artifacts: (1) a concise baseline audit with pinned commit; (2) a minimal patch set with no history destruction; (3) unit/regression tests for every new retrieval/adaptation/model-call invariant; (4) a visual fixture set at every phase; (5) reproducible manifests with exact code/data/config hashes; (6) a held-out paired D/geometry/lookup ablation report; (7) a final decision memo stating **supported**, **negative**, **underpowered** or **blocked** findings without implying completion of unresolved proofs.

Disallow: changing ground-truth labels after test evaluation; treating lack of a library witness as proof of infeasibility; replaying exact joint paths at translated locations without adaptation; measuring retrieval by local similarity alone; computing original-target blocker loss on unlabeled intermediate scenes; training a destination/order/obstacle-feasibility head; introducing L_{updated}; calling the D model with `target=A` in the new production policy; claiming conditional Boltzmann weights are a discovered directed causal graph; using learned predictions as final success certification; leaking test scene derivatives into training; modifying legacy baselines to make the new method appear stronger.

11. Repo map, source basis and versioning

Core model and guidance: `d_blockers.py` implements the pairless/paired energy, likelihood and intervention-union training labels. `p17_d.py` implements current dynamic D guidance, including the temporary-target recursion to **exclude only in the new policy**.

Motion retrieval and verification: `p14_sweep_index.py` stores real witness provenance, local patches, absolute-cell keys and swept occupancy; `p15_lookup.py` implements motion modes and leakage-safe transfer tests; `p16_motion.py` certifies actual current-scene trajectories, robot transit, online fallbacks and independent replay. `p17_domain.py` and `p17_search.py` define candidate movers/slots and bounded finite search.

Archived evidence: `responses-phases/0095*` reports Dp17 training, labels and calibration; `responses-phases/0098*` reports the 90-root prospective D evaluation and its negative/underpowered guidance verdict; `responses-phases/0075*` reports held-out swept-motion transfer and the value of multiple modes. The actual report filenames remain unchanged in the repository.

Repository: https://github.com/Narendhiranv04/latent-causal-tamp. Previous uploaded methodology v1.7 and implementation guide v1.9 principally specify the historical A/B/C comparison and should be treated as **historical or parallel scope**, not overwritten as though this proposed D extension has already passed its tests.

Change policy: Version this file as `approachD_methodology_v1_0`; implement in a separate branch; freeze and preserve P17 comparison arms; make each gate independently runnable; do not proceed beyond a failed gate by reinterpreting `UNKNOWN` as a negative. The initial handoff prompt is supplied separately in `approachD_initial_prompt.md`.